

ml-lab-ass-3

July 18, 2025

1 Assignment 3

1.1 Step 1: Import Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.metrics import confusion_matrix, accuracy_score

# Alternative fallback (not recommended for new projects)
from keras.models import Sequential
from keras.layers import Dense
```

1.2 Step 2: Load the Dataset

```
[2]: df = pd.read_csv('Churn_Modelling.csv') # Replace with actual filename
df.head()
```

```
[2]:
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	\
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	
3	1	0.00	2	0	0	
4	2	125510.82	1	1	1	

	EstimatedSalary	Exited
--	-----------------	--------

0	101348.88	1
1	112542.58	0
2	113931.57	1
3	93826.63	0
4	79084.10	0

1.3 Step 3: Feature Selection

1.3.1 3.1 Drop irrelevant columns:

```
[3]: df = df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)
```

1.3.2 3.2 Encode categorical columns:

```
[4]: # Label encode 'Gender'
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender']) # Male=1, Female=0

# One-hot encode 'Geography'
df = pd.get_dummies(df, columns=['Geography'], drop_first=True)
```

1.4 Step 4: Define Features and Target

```
[5]: X = df.drop('Exited', axis=1)
y = df['Exited']
```

1.5 Step 5: Split into Train and Test Sets

```
[6]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42)
```

1.6 Step 6: Normalize the Data

```
[7]: scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

1.7 Step 7: Build the ANN Model

```
[8]: model = Sequential()
model.add(Dense(16, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid')) # Output layer for binary
↳ classification
```

```
C:\Users\Mehatab Sanadi\.conda\envs\myenv\lib\site-  
packages\keras\src\layers\core\dense.py:87: UserWarning: Do not pass an  
`input_shape`/`input_dim` argument to a layer. When using Sequential models,  
prefer using an `Input(shape)` object as the first layer in the model instead.  
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

1.8 Step 8: Compile and Train the Model

```
[9]: model.compile(optimizer='adam', loss='binary_crossentropy',  
    ↪ metrics=['accuracy'])  
history = model.fit(X_train, y_train, epochs=50, batch_size=32,  
    ↪ validation_split=0.2)
```

```
Epoch 1/50  
200/200          1s 1ms/step -  
accuracy: 0.5104 - loss: 0.6815 - val_accuracy: 0.7987 - val_loss: 0.4897  
Epoch 2/50  
200/200          0s 701us/step -  
accuracy: 0.7936 - loss: 0.4951 - val_accuracy: 0.7987 - val_loss: 0.4731  
Epoch 3/50  
200/200          0s 728us/step -  
accuracy: 0.7994 - loss: 0.4748 - val_accuracy: 0.7987 - val_loss: 0.4634  
Epoch 4/50  
200/200          0s 619us/step -  
accuracy: 0.8005 - loss: 0.4668 - val_accuracy: 0.7987 - val_loss: 0.4551  
Epoch 5/50  
200/200          0s 619us/step -  
accuracy: 0.7952 - loss: 0.4647 - val_accuracy: 0.7987 - val_loss: 0.4491  
Epoch 6/50  
200/200          0s 606us/step -  
accuracy: 0.7950 - loss: 0.4641 - val_accuracy: 0.7987 - val_loss: 0.4434  
Epoch 7/50  
200/200          0s 628us/step -  
accuracy: 0.7880 - loss: 0.4624 - val_accuracy: 0.7987 - val_loss: 0.4361  
Epoch 8/50  
200/200          0s 550us/step -  
accuracy: 0.7946 - loss: 0.4520 - val_accuracy: 0.8138 - val_loss: 0.4302  
Epoch 9/50  
200/200          0s 628us/step -  
accuracy: 0.8040 - loss: 0.4451 - val_accuracy: 0.8206 - val_loss: 0.4253  
Epoch 10/50  
200/200          0s 685us/step -  
accuracy: 0.8065 - loss: 0.4415 - val_accuracy: 0.8188 - val_loss: 0.4204  
Epoch 11/50  
200/200          0s 926us/step -  
accuracy: 0.8086 - loss: 0.4373 - val_accuracy: 0.8263 - val_loss: 0.4163  
Epoch 12/50  
200/200          0s 680us/step -
```

accuracy: 0.8104 - loss: 0.4349 - val_accuracy: 0.8325 - val_loss: 0.4136
 Epoch 13/50
 200/200 0s 643us/step -
 accuracy: 0.8221 - loss: 0.4253 - val_accuracy: 0.8331 - val_loss: 0.4077
 Epoch 14/50
 200/200 0s 681us/step -
 accuracy: 0.8213 - loss: 0.4185 - val_accuracy: 0.8313 - val_loss: 0.4027
 Epoch 15/50
 200/200 0s 677us/step -
 accuracy: 0.8359 - loss: 0.3973 - val_accuracy: 0.8313 - val_loss: 0.4006
 Epoch 16/50
 200/200 0s 635us/step -
 accuracy: 0.8345 - loss: 0.3978 - val_accuracy: 0.8331 - val_loss: 0.3925
 Epoch 17/50
 200/200 0s 603us/step -
 accuracy: 0.8315 - loss: 0.4071 - val_accuracy: 0.8350 - val_loss: 0.3866
 Epoch 18/50
 200/200 0s 677us/step -
 accuracy: 0.8317 - loss: 0.3961 - val_accuracy: 0.8356 - val_loss: 0.3820
 Epoch 19/50
 200/200 0s 654us/step -
 accuracy: 0.8334 - loss: 0.3958 - val_accuracy: 0.8375 - val_loss: 0.3790
 Epoch 20/50
 200/200 0s 570us/step -
 accuracy: 0.8397 - loss: 0.3749 - val_accuracy: 0.8400 - val_loss: 0.3750
 Epoch 21/50
 200/200 0s 814us/step -
 accuracy: 0.8425 - loss: 0.3772 - val_accuracy: 0.8431 - val_loss: 0.3706
 Epoch 22/50
 200/200 0s 620us/step -
 accuracy: 0.8471 - loss: 0.3606 - val_accuracy: 0.8375 - val_loss: 0.3671
 Epoch 23/50
 200/200 0s 690us/step -
 accuracy: 0.8484 - loss: 0.3617 - val_accuracy: 0.8456 - val_loss: 0.3639
 Epoch 24/50
 200/200 0s 659us/step -
 accuracy: 0.8570 - loss: 0.3578 - val_accuracy: 0.8431 - val_loss: 0.3610
 Epoch 25/50
 200/200 0s 607us/step -
 accuracy: 0.8528 - loss: 0.3602 - val_accuracy: 0.8456 - val_loss: 0.3591
 Epoch 26/50
 200/200 0s 654us/step -
 accuracy: 0.8545 - loss: 0.3570 - val_accuracy: 0.8450 - val_loss: 0.3577
 Epoch 27/50
 200/200 0s 688us/step -
 accuracy: 0.8512 - loss: 0.3638 - val_accuracy: 0.8456 - val_loss: 0.3571
 Epoch 28/50
 200/200 0s 550us/step -

accuracy: 0.8575 - loss: 0.3433 - val_accuracy: 0.8487 - val_loss: 0.3547
 Epoch 29/50
 200/200 0s 704us/step -
 accuracy: 0.8467 - loss: 0.3543 - val_accuracy: 0.8512 - val_loss: 0.3571
 Epoch 30/50
 200/200 0s 630us/step -
 accuracy: 0.8590 - loss: 0.3464 - val_accuracy: 0.8506 - val_loss: 0.3508
 Epoch 31/50
 200/200 0s 562us/step -
 accuracy: 0.8596 - loss: 0.3432 - val_accuracy: 0.8500 - val_loss: 0.3493
 Epoch 32/50
 200/200 0s 615us/step -
 accuracy: 0.8543 - loss: 0.3476 - val_accuracy: 0.8506 - val_loss: 0.3485
 Epoch 33/50
 200/200 0s 938us/step -
 accuracy: 0.8496 - loss: 0.3579 - val_accuracy: 0.8519 - val_loss: 0.3494
 Epoch 34/50
 200/200 0s 704us/step -
 accuracy: 0.8599 - loss: 0.3361 - val_accuracy: 0.8512 - val_loss: 0.3491
 Epoch 35/50
 200/200 0s 531us/step -
 accuracy: 0.8606 - loss: 0.3435 - val_accuracy: 0.8512 - val_loss: 0.3479
 Epoch 36/50
 200/200 0s 628us/step -
 accuracy: 0.8546 - loss: 0.3481 - val_accuracy: 0.8525 - val_loss: 0.3492
 Epoch 37/50
 200/200 0s 574us/step -
 accuracy: 0.8531 - loss: 0.3451 - val_accuracy: 0.8519 - val_loss: 0.3472
 Epoch 38/50
 200/200 0s 628us/step -
 accuracy: 0.8622 - loss: 0.3352 - val_accuracy: 0.8425 - val_loss: 0.3535
 Epoch 39/50
 200/200 0s 628us/step -
 accuracy: 0.8580 - loss: 0.3365 - val_accuracy: 0.8519 - val_loss: 0.3454
 Epoch 40/50
 200/200 0s 628us/step -
 accuracy: 0.8549 - loss: 0.3454 - val_accuracy: 0.8519 - val_loss: 0.3447
 Epoch 41/50
 200/200 0s 555us/step -
 accuracy: 0.8538 - loss: 0.3460 - val_accuracy: 0.8494 - val_loss: 0.3442
 Epoch 42/50
 200/200 0s 550us/step -
 accuracy: 0.8602 - loss: 0.3346 - val_accuracy: 0.8531 - val_loss: 0.3447
 Epoch 43/50
 200/200 0s 825us/step -
 accuracy: 0.8620 - loss: 0.3361 - val_accuracy: 0.8531 - val_loss: 0.3445
 Epoch 44/50
 200/200 0s 578us/step -

```

accuracy: 0.8610 - loss: 0.3324 - val_accuracy: 0.8500 - val_loss: 0.3444
Epoch 45/50
200/200          0s 628us/step -
accuracy: 0.8639 - loss: 0.3301 - val_accuracy: 0.8487 - val_loss: 0.3444
Epoch 46/50
200/200          0s 628us/step -
accuracy: 0.8571 - loss: 0.3410 - val_accuracy: 0.8544 - val_loss: 0.3433
Epoch 47/50
200/200          0s 628us/step -
accuracy: 0.8612 - loss: 0.3349 - val_accuracy: 0.8525 - val_loss: 0.3434
Epoch 48/50
200/200          0s 628us/step -
accuracy: 0.8658 - loss: 0.3276 - val_accuracy: 0.8519 - val_loss: 0.3423
Epoch 49/50
200/200          0s 628us/step -
accuracy: 0.8547 - loss: 0.3364 - val_accuracy: 0.8537 - val_loss: 0.3430
Epoch 50/50
200/200          0s 550us/step -
accuracy: 0.8638 - loss: 0.3316 - val_accuracy: 0.8519 - val_loss: 0.3454

```

1.9 Step 9: Evaluate the Model

```

[10]: # Predict
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int)

# Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {acc:.4f}")

```

```

63/63          0s 741us/step
Model Accuracy: 0.8590

```

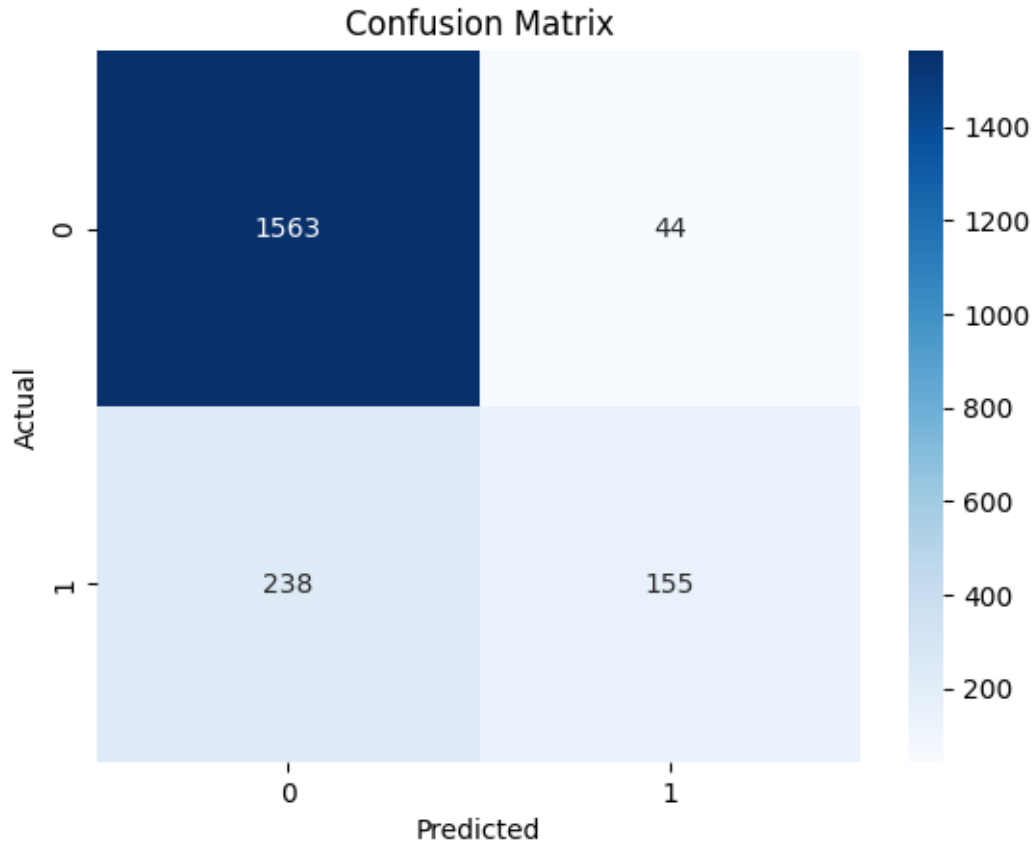
1.10 Step 10: Confusion Matrix

```

[11]: cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



1.11 Observation

1. True Negatives (TN = 1563): Model correctly predicted non-churn customers.
2. True Positives (TP = 155): Model correctly predicted churners.
3. False Positives (FP = 44): Non-churners incorrectly predicted as churners (Type I error).
4. False Negatives (FN = 238): Churners incorrectly predicted as non-churners (Type II error).

Conclusion The model performs well in identifying customers who will stay but struggles with detecting churners, shown by a high number of False Negatives. Improving recall for churn predictions is essential, as missing potential churners can lead to lost opportunities for retention.